ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF APPLIED SCIENCE, FAS, HCMUT

Academic Year 2025–2026

Major Project Report

COURSE: Linear Algebra (MT1007)

Discrete Fourier Transform (DFT–FFT) and Audio Denoising

Group information	Group – Section: **6677 – CC04**
Lecturer	Dr. **Đặng Thị Kim Nhung**
Topic source	Assignment file `dft_fft_audio_denoising.pdf`
Report orientation	Theory – Algorithm – Implementation – Experiment – Evaluation

Ho Chi Minh City, 2026

Group Member Information

No.	Full name	Student ID	Participation	Main responsibility
1	Trần Anh Khoa	2551978	100%	Cover page, introduction, and report formatting
2	Nguyễn Thanh Diễm Ngọc	2550932	100%	Fourier theory and continuous-to-discrete derivation
3	Trần Hà My	2550923	100%	DFT formula analysis and matrix interpretation
4	Nguyễn Đoàn Phú Khang	2550296	100%	FFT idea, complexity analysis, and algorithm description
5	Hoàng Trọng Hiếu	2550284	100%	Audio preprocessing and experimental setup
6	Nguyễn Viết Nhật Minh	2550148	100%	Python implementation and denoising pipeline
7	Võ Hoàng Anh Kiệt	2550131	100%	Plot generation and result evaluation
8	Phạm Trương Hữu Tín	2510243	100%	Conclusion, references, and final review

Contents

Group Member Information **1**

1 Introduction **4**

1.1 Objectives 5

1.2 Scope of the Project 5

1.3 Motivation and Guiding Questions 6

1.4 Report Structure 6

2 Theoretical Foundation **7**

2.1 From Fourier Series to the Fourier Transform 7

2.2 Signal Discretization and the Birth of the DFT 7

2.3 Matrix Interpretation of the DFT 9

2.4 Relevant Theorems and Their Roles 10

2.5 Why FFT is Needed 12

2.6 Origin and Core Idea of the Fast Fourier Transform 13

2.7 Radix-2 Cooley–Tukey Derivation 14

3 Analysis of the DFT Formula **15**

3.1 Meaning of Each Quantity 15

3.2 Time Domain and Frequency Domain 15

3.3 Implications for Audio Denoising 16

4 Algorithmic Design and Implementation Orientation **17**

4.1 Direct DFT Algorithm 17

4.2 Chunk Processing and Overlap-Add Reconstruction 18

4.3 FFT Idea and Radix-2 Cooley–Tukey Algorithm 19

4.4 Complexity Comparison 21

4.5 Implementation Constraints from the Assignment 22

5 Audio Denoising Pipeline 22

5.1 Data Preparation 22

5.2 Step 1: Transform to the Frequency Domain 22

5.3 Step 2: Identify and Attenuate Noise Components 23

5.4 Step 3: Signal Reconstruction 23

5.5 Expected Figures and Tables 24

6 Experimental Results Template 24

6.1 Experiment Setup 24

6.2 Program Output and Visualization 24

6.3 Result Discussion 26

6.4 Comparison Between DFT-Based and FFT-Based Results 27

6.5 Evaluation Note 28

6.6 Project Deliverables 28

7 Conclusion and Future Work 29

Appendix: Requirement Checklist 31

1. Introduction

Every audio recording is a mixture of intention and interference. A speaker may want to record a clear sentence, a musician may want to preserve the tone of an instrument, or a researcher may want to analyze a short acoustic signal. However, the microphone never captures only the desired sound. It also records the room, the fan, the electrical hum, the small vibration of the device, and sometimes even random fluctuations that are not meaningful to the listener. The central question of this report is therefore both practical and mathematical: *how can we separate useful sound from unwanted noise when both are mixed into the same sequence of numbers?*

At first glance, this seems difficult because a digital audio file is simply a long list of amplitude values. In the time domain, noise and useful signal may overlap at the same sample indices, making direct separation almost impossible. A sudden fluctuation in the waveform does not immediately tell us whether it comes from the desired sound or from noise. Fourier analysis provides a powerful change of perspective. Instead of looking only at how amplitude changes over time, it asks what oscillatory components are hidden inside the signal. Once the signal is expressed as a combination of frequencies, some noise patterns become easier to identify: a low-frequency hum may appear as a concentrated spectral peak, while random background noise may spread as many weaker components across the spectrum.

This project studies the mathematical basis and practical application of the Discrete Fourier Transform (DFT) and the Fast Fourier Transform (FFT) for the audio denoising problem. The report is built directly from the requirements stated in the assignment file `dft_fft_audio_denoising.pdf`, while preserving the overall organization commonly used in a Bach Khoa major-project report. The focus is not merely to use a library command that hides the computation, but to understand what the transform does, why it works, and how its coefficients can be modified to reconstruct a cleaner signal.

The topic is also a natural application of linear algebra. A finite audio segment can be viewed as a vector in a finite-dimensional vector space. The DFT is then a linear transformation that changes the coordinate system from the standard time basis to a complex exponential frequency basis. In that new basis, filtering is no longer an abstract operation: it becomes the act of attenuating selected coordinates. This interpretation connects the computational experiment to core ideas from the course,

including vectors, matrices, complex numbers, linear transformations, and algorithmic complexity.

From an engineering viewpoint, the project is deliberately modest but meaningful. The implemented program processes a noisy signal, computes the DFT directly from the defining formula, applies a magnitude-based filtering rule, and reconstructs the signal by the inverse transform. Because the direct DFT has quadratic complexity, the signal cannot be treated as one huge vector in a naive way. The implementation therefore processes the audio in overlapping chunks and recombines them carefully. This practical compromise makes the project closer to real signal processing: mathematical formulas must be adapted to computational limits without losing their theoretical meaning.

1.1. Objectives

The report aims to satisfy the following objectives:

- Establish the theoretical background from Fourier series to the Fourier transform, then to signal discretization and the DFT.
- Explain the meaning of each quantity in the DFT formula

$$X_k = \sum_{n=0}^{N-1} x[n] e^{-2\pi i k n / N}.$$

- Clarify the difference between the time domain and the frequency domain, especially for audio denoising.
- Present the main algorithmic idea of direct DFT and FFT without depending on black-box built-in transforms for the core results.
- Demonstrate a denoising pipeline: spectrum analysis, noise attenuation, and signal reconstruction.

1.2. Scope of the Project

- Object: real recorded audio files with additive noise, preferably mono or low-polyphony signals.

- Core mathematical tools: complex numbers, vector spaces, orthogonality, matrix representation, and computational complexity.
- Expected outputs: waveform plots before/after denoising, spectrum plots before/after filtering, algorithm explanation, and implementation evidence.

1.3. Motivation and Guiding Questions

The report is guided by several concrete questions. First, what does it mean for an audio sample sequence to contain a frequency? Second, why does the DFT coefficient X_k become complex even when the input audio values are real? Third, how can the magnitude $|X_k|$ be used as evidence that a frequency component is strong or weak? Finally, what difficulties appear when the mathematical formula is implemented on a computer with finite memory and limited processing speed?

Answering these questions helps prevent a common misunderstanding: denoising is not simply deleting random samples from the waveform. Instead, the procedure is a structured transformation-filtering-reconstruction pipeline. The original waveform is first represented in a frequency basis, the spectrum is modified according to a rule, and the time-domain signal is then rebuilt using the inverse transform. If the spectral modification is too aggressive, useful signal components may be lost; if it is too weak, noise remains. The project therefore illustrates both the power and the limitations of frequency-domain processing.

1.4. Report Structure

This report is organized into five main parts: theoretical background, analysis of the DFT formula, algorithmic implementation, experimental denoising procedure, and concluding remarks. The structure is intentionally straightforward so that the mathematical logic remains central.

2. Theoretical Foundation

2.1. From Fourier Series to the Fourier Transform

Fourier analysis begins with the idea that a sufficiently regular periodic signal can be represented as a sum of sinusoidal components. For a continuous periodic function $f(t)$ with period T, the classical Fourier series has the form

$$f(t) = \sum_{m=-\infty}^{\infty} c_m e^{i2\pi mt/T}, \qquad c_m = \frac{1}{T}\int_0^T f(t)e^{-i2\pi mt/T}\,dt.$$

Here, $f(t)$ is the continuous-time signal, T is the period, m is an integer harmonic index, and c_m is the Fourier coefficient of the m-th harmonic.

When the period tends to infinity, the frequency variable becomes continuous and the Fourier series transitions into the continuous Fourier transform. This viewpoint explains why the Fourier transform decomposes a signal into elementary oscillatory modes.

The important conceptual point is orthogonality. Complex exponentials with different frequencies behave like independent directions in a vector space of functions. When a signal is projected onto one of these directions, the result measures how much of that oscillation is present in the signal. This is similar to decomposing a vector into components along perpendicular axes, except that the axes are sinusoidal waves rather than geometric coordinate directions. For audio, this means that a complicated waveform can be interpreted as a mixture of simpler tones.

In continuous theory, the transform uses integrals because the signal is defined over a continuum of time. In digital processing, however, computers store finite sequences. This transition from an integral over time to a finite summation over samples is exactly where the DFT becomes relevant. The DFT keeps the same decomposition idea but adapts it to the finite-dimensional setting required by recorded audio.

2.2. Signal Discretization and the Birth of the DFT

In digital systems, an audio signal is sampled at discrete time instants. If the sampling frequency is F_s and the observation window contains N samples, the signal becomes a

finite vector

$$x = \big(x[0], x[1], \ldots, x[N-1]\big)^T.$$

Here, x is the finite signal vector, $x[n]$ is the amplitude value of the n-th sample, $n = 0, 1, \ldots, N-1$ is the sample index, and N is the total number of samples in the analyzed block.

The finite and discrete nature of digital data leads naturally to the Discrete Fourier Transform:

$$X_k = \sum_{n=0}^{N-1} x[n] e^{-2\pi i k n/N}, \qquad k = 0, 1, \ldots, N-1.$$

Here, X_k is the DFT coefficient at frequency bin k, $x[n]$ is the n-th time-domain sample, N is the block length, and $e^{-2\pi i k n/N}$ is the complex exponential basis component.

The inverse transform is given by

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X_k e^{2\pi i k n/N}.$$

Here, $x[n]$ is the reconstructed time-domain sample, X_k is the k-th DFT coefficient, and the factor $1/N$ normalizes the inverse transform.

The value F_s determines how many samples are captured per second. If $F_s = 8000$ Hz, then the signal contains 8000 amplitude measurements every second. For the nonnegative-frequency part of the spectrum, the bin index k corresponds to a physical frequency approximately equal to

$$f_k = \frac{kF_s}{N}, \qquad k = 0, 1, \ldots, \frac{N}{2}.$$

Here, f_k is the physical frequency represented by bin k, F_s is the sampling frequency, and N is the number of samples in the analyzed block.

Thus, increasing N improves frequency resolution because the spacing

$$\Delta f = \frac{F_s}{N}$$

becomes smaller. Here, Δf is the frequency spacing between two adjacent DFT bins. However, a larger N also increases computational cost. This trade-off between frequency resolution and computational feasibility appears directly in our implementation: long signals provide more spectral detail, but direct DFT computation becomes expensive.

For real-valued audio signals, the DFT spectrum has conjugate symmetry: the information in the second half of the spectrum mirrors the first half. This is why the plotted spectra often focus on the nonnegative-frequency part of the spectrum, from bin 0 to bin $N/2$. These bins represent nonnegative frequencies up to the Nyquist frequency $F_s/2$, which is the highest frequency that can be represented without aliasing under the sampling theorem.

2.3. Matrix Interpretation of the DFT

Define the matrix $W \in \mathbb{C}^{N\times N}$ by

$$W_{k,n} = e^{-2\pi ikn/N}.$$

Here, W is the DFT matrix, $W_{k,n}$ is the entry in row k and column n, k is the frequency-bin index, and n is the time-sample index.

Then the DFT can be written compactly as

$$X = Wx.$$

Here, x is the time-domain signal vector, W is the DFT matrix, and X is the frequency-domain vector whose entries are the coefficients X_k. This representation is important in linear algebra because it shows that the DFT is a linear transformation from the time-domain vector space to the frequency-domain vector space.

The rows of W contain sampled complex exponentials. Multiplying W by x computes all inner products between the signal and these oscillatory basis vectors. Therefore, the matrix form is not only a compact notation; it reveals the structure of the computation. Each output coefficient is a coordinate of the original signal in a frequency-oriented coordinate system.

The inverse DFT can also be expressed in matrix form. Since the DFT matrix satisfies a scaled unitary relation

$$W^H W = NI,$$

the inverse is

$$W^{-1} = \frac{1}{N} W^H.$$

Here, W^H denotes the conjugate transpose of W, I is the $N \times N$ identity matrix, and N is the number of samples. This relation explains why the inverse formula resembles the forward formula but uses the opposite sign in the exponential and includes the scaling factor $1/N$.

2.4. Relevant Theorems and Their Roles

After introducing the DFT matrix form, it is useful to state the main mathematical properties that justify this transformation. These properties explain why the DFT basis can separate frequency components, why the inverse transform can reconstruct the signal, and why real audio signals produce symmetric spectra.

The following properties are standard results of the Discrete Fourier Transform. In this project, they are used to connect the audio denoising process with the linear algebra ideas of orthogonality, basis representation, and invertible linear transformations.

Orthogonality of the DFT Basis

For integers $k, m \in \{0, 1, \ldots, N-1\}$, the sampled complex exponential vectors satisfy

$$\sum_{n=0}^{N-1} e^{2\pi i (k-m)n/N} = \begin{cases} N, & k = m, \\ 0, & k \neq m. \end{cases}$$

Here, k and m are two frequency-bin indices, n is the sample index, and N is the block length.

This property means that different DFT frequency components are orthogonal over one

complete block of N samples. In linear algebra terms, they behave like independent basis directions. Therefore, the coefficient X_k measures how strongly the original signal contains the k-th frequency component.

For the denoising task, this property is important because it explains why the signal can be analyzed by frequency. In the time domain, useful sound and noise are mixed sample by sample. In the frequency domain, the signal is represented by coefficients along different frequency directions, so selected components can be attenuated more systematically.

Invertibility of the DFT

If the DFT coefficients of a finite sequence $x[0], x[1], \ldots, x[N-1]$ are defined by

$$X_k = \sum_{n=0}^{N-1} x[n] e^{-2\pi i k n/N}, \qquad k = 0, 1, \ldots, N-1,$$

then the original sequence can be reconstructed by

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X_k e^{2\pi i k n/N}, \qquad n = 0, 1, \ldots, N-1.$$

Here, $x[n]$ is the time-domain sample, X_k is the DFT coefficient, n is the sample index, k is the frequency-bin index, and N is the number of samples.

Equivalently, if W is the DFT matrix, then

$$W^{-1} = \frac{1}{N} W^H.$$

Here, W^H denotes the conjugate transpose of W. This shows that the DFT is an invertible linear transformation from the time domain to the frequency domain.

This result supports the main structure of the project. In mathematical notation, the denoising process can be summarized as

$$x_{\text{noisy}}[n] \xrightarrow{\text{DFT}} X_k \xrightarrow{\text{filtering}} \widetilde{X}_k \xrightarrow{\text{IDFT}} \widetilde{x}[n].$$

Here, $x_{\text{noisy}}[n]$ is the noisy input signal, X_k is its DFT coefficient, $\widetilde{X}_k$ is the filtered

frequency coefficient, and $\widetilde{x}[n]$ is the reconstructed denoised signal. Without the inverse DFT, the filtered frequency coefficients could not be converted back into an audio waveform.

Conjugate Symmetry for Real Signals

If the input signal $x[n]$ is real-valued, then its DFT coefficients satisfy

$$X_{N-k} = \overline{X_k}, \qquad k = 1, 2, \ldots, N-1.$$

Here, X_{N-k} is the coefficient symmetric to X_k in the DFT output, and $\overline{X_k}$ denotes the complex conjugate of X_k.

This property means that the DFT spectrum of a real-valued signal has conjugate symmetry. The second half of the spectrum mirrors the first half, so it does not provide completely new frequency information.

Since ordinary audio samples are real numbers, this property explains why the experiment mainly focuses on the nonnegative frequency range from 0 to the Nyquist frequency $F_s/2$. It also makes the spectrum plots easier to interpret.

Connection to the Project

These properties justify the denoising pipeline used in this report. Orthogonality explains why the DFT can separate a signal into frequency components. Invertibility explains why the filtered spectrum can be transformed back into a time-domain waveform. Conjugate symmetry explains the mirrored structure of the spectrum for real audio signals.

Therefore, the denoising method is not only a programming procedure. It is based on representing an audio signal in a frequency basis, modifying selected coefficients, and reconstructing the signal through an inverse linear transformation.

2.5. Why FFT is Needed

Computing the DFT formula directly requires approximately $O(N^2)$ complex multiplications/additions, where N is the number of samples in the analyzed block. For large

audio signals, this cost becomes expensive. The FFT is not a different transform; it is an efficient algorithm that exploits symmetries and factorization patterns in the DFT matrix to reduce the complexity to approximately $O(N \log N)$.

The distinction between DFT and FFT is essential. The DFT is the mathematical transformation: it defines the exact relationship between $x[n]$ and X_k. The FFT is an algorithmic strategy for evaluating that same transformation faster. Therefore, using FFT instead of direct DFT does not change the frequency-domain representation; it only computes the same coefficients more efficiently. In this assignment, the main implementation intentionally uses the direct formula to make the mathematics transparent. Nevertheless, the complexity discussion motivates why practical audio software normally uses FFT-based routines when processing long recordings in real time.

2.6. Origin and Core Idea of the Fast Fourier Transform

The Fast Fourier Transform became widely known through the Cooley–Tukey algorithm, published by James W. Cooley and John W. Tukey in 1965. However, the central idea of reducing Fourier-transform computation by factorization had appeared earlier in mathematical work, including ideas attributed to Gauss. The importance of the Cooley–Tukey formulation is that it made the method systematic, practical, and highly influential for digital computation.

The core observation is that the DFT formula contains repeated structure. Direct DFT computation evaluates

$$X_k = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \qquad W_N = e^{-2\pi i/N},$$

for every $k = 0, 1, \ldots, N-1$. Here, W_N is the primitive N-th root of unity. If this formula is computed naively, each of the N output coefficients requires a summation over N samples, so the total cost is $O(N^2)$.

The FFT does not approximate the DFT. Instead, it reorganizes the same summation so that repeated terms are reused. For the radix-2 Cooley–Tukey algorithm, the input length is assumed to be a power of two, $N = 2^m$. This assumption allows the signal to be split repeatedly into even-index and odd-index subsequences.

2.7. Radix-2 Cooley–Tukey Derivation

Starting from the DFT formula,

$$X_k = \sum_{n=0}^{N-1} x[n] W_N^{kn},$$

we separate the sum into even and odd indices. Let $n = 2r$ for even indices and $n = 2r + 1$ for odd indices. Then

$$\begin{aligned} X_k &= \sum_{r=0}^{N/2-1} x[2r] W_N^{k(2r)} + \sum_{r=0}^{N/2-1} x[2r+1] W_N^{k(2r+1)} \\ &= \sum_{r=0}^{N/2-1} x[2r] W_{N/2}^{kr} + W_N^k \sum_{r=0}^{N/2-1} x[2r+1] W_{N/2}^{kr}. \end{aligned}$$

The equality uses the identity $W_N^{2kr} = W_{N/2}^{kr}$. Now define

$$E_k = \sum_{r=0}^{N/2-1} x[2r] W_{N/2}^{kr}, \qquad O_k = \sum_{r=0}^{N/2-1} x[2r+1] W_{N/2}^{kr}.$$

Here, E_k is the DFT of the even-index subsequence and O_k is the DFT of the odd-index subsequence. Therefore,

$$X_k = E_k + W_N^k O_k.$$

Because the DFT output has periodic structure, the second half of the spectrum can be computed at the same time:

$$X_{k+N/2} = E_k - W_N^k O_k, \qquad k = 0, 1, \ldots, \frac{N}{2} - 1.$$

These two formulas are called the **butterfly operation**. They combine two smaller DFTs into one larger DFT using only additions, subtractions, and multiplication by twiddle factors W_N^k.

The algorithm then applies the same decomposition recursively. A DFT of length N becomes two DFTs of length $N/2$, then four DFTs of length $N/4$, and so on until the base case of length 1. Since there are $\log_2 N$ recursion levels and each level performs about N butterfly operations, the total complexity is

$$O(N \log_2 N).$$

For the block size used in this project, $N = 1024 = 2^{10}$. Direct DFT needs on the order of $1024^2 = 1{,}048{,}576$ operations per block, while radix-2 FFT needs on the order of $1024 \cdot 10 = 10{,}240$ operations. This is why the FFT version is much faster while producing the same frequency-domain coefficients in exact arithmetic.

3. Analysis of the DFT Formula

3.1. Meaning of Each Quantity

The required formula in the assignment is

$$X_k = \sum_{n=0}^{N-1} x[n] e^{-2\pi i k n/N}.$$

Table 1: Interpretation of the quantities in the DFT formula

Quantity	Meaning
$x[n]$	The value of the signal at the n-th sample in the time domain.
N	The total number of samples in the analyzed frame or signal segment.
k	The frequency-bin index, associated with a discrete frequency value.
X_k	The complex coefficient representing amplitude and phase information of the k-th frequency component.
$\lvert X_k \rvert$	The magnitude of X_k, used to measure the strength of the corresponding frequency component.
$\arg(X_k)$	The phase angle of the frequency component, relevant for accurate reconstruction.

3.2. Time Domain and Frequency Domain

The time domain describes how the audio amplitude changes with time. In this view, one sees the raw waveform, transient events, and global energy evolution. However, it is difficult to separate noise from useful information when both overlap in time.

The frequency domain describes how signal energy is distributed across oscillation frequencies. This view is essential for denoising because certain types of noise appear

as localized peaks or broad bands that can be identified and attenuated more clearly than in the time domain.

For example, an electrical hum may be difficult to identify from the waveform alone because it is added to every part of the signal. In the frequency domain, however, it may appear around a narrow low-frequency region. Conversely, random noise often does not concentrate at a single bin; instead, it raises the background level of the spectrum. These two cases motivate different filtering rules. A notch filter is suitable for narrow-band interference, while magnitude thresholding or spectral subtraction is more suitable for distributed noise.

In audio denoising, neither domain is universally better. The time domain preserves temporal localization: it tells us when an event occurs. The frequency domain reveals oscillatory composition: it tells us which rates of vibration are present. A good signal-processing method often moves between the two domains, using each representation where it is most informative.

3.3. Implications for Audio Denoising

In the denoising problem, the workflow typically follows three mathematical steps:

1. Transform the observed noisy signal from the time domain to the frequency domain.
2. Identify noise-dominated frequency components and attenuate or remove them.
3. Apply the inverse transform to reconstruct a cleaner time-domain signal.

This explains why Fourier analysis is highly effective: it converts a difficult separation problem in time into a more structured filtering problem in frequency.

4. Algorithmic Design and Implementation Orientation

4.1. Direct DFT Algorithm

The direct implementation follows the formula exactly. For each output index k, the algorithm sums all contributions from the input samples:

1. Initialize $X_k = 0$.
2. For each $n = 0, 1, \ldots, N-1$, compute the complex exponential $e^{-2\pi ikn/N}$.
3. Accumulate the product $x[n]e^{-2\pi ikn/N}$.

The following Python code is the core implementation used in the project. The functions `manual_dft` and `manual_idft` are implemented directly from the mathematical formulas instead of using built-in functions such as `fft()` or `ifft()`. The denoising function then applies a simple magnitude-threshold rule in the frequency domain.

```
def manual_dft(x):
    N = len(x)
    n = np.arange(N)
    k = n.reshape((N, 1))
    exponent = -2j * np.pi * k * n / N
    X = np.dot(np.exp(exponent), x)
    return X

def manual_idft(X):
    N = len(X)
    k = np.arange(N)
    n = k.reshape((N, 1))
    exponent = 2j * np.pi * k * n / N
    x_reconstructed = np.dot(np.exp(exponent), X) / N
    return np.real(x_reconstructed)

def denoise(X, threshold_ratio=0.1):
    magnitudes = np.abs(X)
    max_mag = np.max(magnitudes)
    X_clean = X.copy()
    X_clean[magnitudes < (threshold_ratio * max_mag)] = 0
```

```
    return X_clean
```

Listing 1: Core implementation of direct DFT, inverse DFT, and magnitude-threshold denoising

In the complete program, the audio signal is processed by overlapping blocks of 1024 samples. Each block is multiplied by a Hanning window, transformed to the frequency domain, filtered by magnitude thresholding, reconstructed by the inverse transform, and finally combined using overlap-add reconstruction. This design keeps the direct DFT computation manageable while reducing discontinuities between adjacent signal blocks.

4.2. Chunk Processing and Overlap-Add Reconstruction

The direct DFT has complexity $O(N^2)$. This means that doubling the number of samples approximately quadruples the number of operations. If the whole audio file were transformed as one long vector, the computation would quickly become too heavy for a simple formula-based implementation. To avoid overloading the machine, the program divides the signal into chunks of 1024 samples. Each chunk is short enough for the direct DFT to run in a reasonable time, but long enough to contain useful local frequency information.

A naive chunk-by-chunk approach creates another problem. If the chunks are processed independently and then simply concatenated, the boundary between two adjacent chunks may contain sudden jumps. These jumps are heard as clicking or artificial noise because the waveform value at the end of one chunk may not match smoothly with the beginning of the next chunk. This is why overlap-add is used. Adjacent chunks overlap by 50%, each chunk is multiplied by a Hanning window, and the overlapping regions are averaged through window normalization. The window gradually fades a chunk in and out, reducing discontinuities at the boundaries.

```
chunk_size = 1024
hop_size = chunk_size // 2
window = np.hanning(chunk_size)

cleaned_data_full = np.zeros(total_samples + chunk_size)
window_sum = np.zeros(total_samples + chunk_size)
steps = range(0, total_samples - chunk_size, hop_size)
```

```
8
9  for start in steps:
10     end = start + chunk_size
11     x_chunk = data[start:end]
12     x_win = x_chunk * window
13
14     max_val = np.max(np.abs(x_win))
15     if max_val > 0:
16         X_chunk = manual_dft(x_win / max_val)
17         X_filt = denoise(X_chunk, threshold_ratio=0.15)
18         x_res = manual_idft(X_filt)
19
20         cleaned_data_full[start:end] += x_res * max_val * window
21         window_sum[start:end] += window**2
22
23 valid_idx = window_sum > 1e-10
24 cleaned_data_full[valid_idx] /= window_sum[valid_idx]
25 final_audio = cleaned_data_full[:total_samples]
```

Listing 2: Main execution loop using Hanning windows and overlap-add reconstruction

This code shows the practical compromise between theory and computation. The transform is still computed directly from the DFT formula, satisfying the mathematical requirement of the assignment. At the same time, the audio is divided into manageable segments so that the quadratic complexity does not make the program impractical. The overlap-add step then repairs the continuity problem introduced by segmentation, producing a reconstructed signal that is smoother than simple chunk concatenation.

4.3. FFT Idea and Radix-2 Cooley–Tukey Algorithm

The Fast Fourier Transform (FFT) is not a different transform from the DFT. It computes the same frequency coefficients X_k, but it organizes the arithmetic more efficiently. Therefore, the DFT-based and FFT-based denoising programs have the same mathematical nature: both transform each audio chunk into the frequency domain, modify spectral coefficients, and reconstruct the signal by an inverse transform. The main difference is computational efficiency.

The FFT implementation in `denoising_by_fft.py` uses the radix-2 Cooley–Tukey algorithm. The term *radix-2* means that the input length must be a power of two,

such as 512, 1024, or 2048. This condition allows the algorithm to repeatedly divide the signal into two equal parts. In this project, the chunk size is $1024 = 2^{10}$, so it fits the radix-2 requirement exactly.

Starting from the DFT formula, the input sequence can be separated into even-index and odd-index samples:

$$x_{even}[r] = x[2r], \qquad x_{odd}[r] = x[2r+1].$$

Substituting this split into the DFT gives

$$X_k = \sum_{r=0}^{N/2-1} x[2r]e^{-2\pi ik(2r)/N} + e^{-2\pi ik/N} \sum_{r=0}^{N/2-1} x[2r+1]e^{-2\pi ik(2r)/N}.$$

The first summation is the DFT of the even-index subsequence, and the second summation is the DFT of the odd-index subsequence multiplied by a rotation factor. This rotation factor

$$W_N^k = e^{-2\pi ik/N}$$

is called the *twiddle factor*. The two smaller spectra are combined through the butterfly relations

$$X_k = E_k + W_N^k O_k, \qquad X_{k+N/2} = E_k - W_N^k O_k.$$

This even–odd decomposition is repeated recursively until the subproblem length becomes one. Since each recursion level performs approximately N operations and there are $\log_2 N$ levels, the total complexity becomes $O(N \log N)$ instead of $O(N^2)$.

Only the two core functions from the FFT implementation are shown below. The remaining denoising steps, such as thresholding, Hanning windowing, and overlap-add reconstruction, follow the same structure as the DFT version.

```
def custom_fft(x):
    x = np.asarray(x, dtype=complex)
    N = len(x)

    if N <= 1:
        return x

    even = custom_fft(x[0::2])
    odd = custom_fft(x[1::2])

```

```
    factor = np.exp(-2j * np.pi * np.arange(N // 2) / N)
    T = factor * odd

    return np.concatenate([even + T, even - T])

def custom_ifft(X):
    X = np.asarray(X, dtype=complex)
    N = len(X)

    X_star = np.conjugate(X)
    fft_star = custom_fft(X_star)
    x = np.conjugate(fft_star) / N

    return np.real(x)
```

Listing 3: Core radix-2 Cooley–Tukey FFT and inverse FFT functions from denoising_by_fft.py

The inverse function uses the conjugate identity

$$\mathrm{IFFT}(X) = \frac{1}{N}\overline{\mathrm{FFT}(\overline{X})}.$$

This is why the code can reuse the forward FFT routine inside `custom_ifft`. Taking the conjugate changes the sign of the imaginary exponent, and dividing by N applies the inverse-transform scaling factor.

4.4. Complexity Comparison

Table 2: Computational comparison between direct DFT and FFT

Method	Complexity	Remark
Direct DFT	$O(N^2)$	Computes every coefficient directly from the definition; transparent but slow for long audio.
Radix-2 FFT	$O(N \log N)$	Computes the same DFT coefficients through recursive even–odd factorization and butterfly operations.

For a chunk of length $N = 1024$, the direct DFT requires on the order of $1024^2 = 1{,}048{,}576$ coefficient-level operations. The radix-2 FFT requires approx-

imately $1024 \log_2(1024) = 1024 \cdot 10 = 10{,}240$ operations. This theoretical reduction explains why FFT is much faster in practice while giving the same frequency-domain representation as the DFT.

4.5. Implementation Constraints from the Assignment

The assignment explicitly requires that the main results must not rely directly on built-in functions such as `fft()` or `ifft()`. Therefore, the report should explain the algorithmic logic before presenting implementation results. Built-in routines may at most be used later for optional cross-checking, not as the main computational engine.

5. Audio Denoising Pipeline

5.1. Data Preparation

The implementation uses a noisy mono audio sample named `sample.wav`. For demonstration and reproducibility, the signal contains two dominant sinusoidal components mixed with low-frequency hum and broadband random noise. The processing script reconstructs the filtered signal and saves it as `sample_cleaned_smooth.wav`.

- File name: `sample.wav`
- Sampling frequency: 8000 Hz
- Number of samples: 9600 samples
- Audio duration: approximately 1.20 seconds
- Channel format: mono
- Noise type: low-frequency hum combined with broadband additive noise

5.2. Step 1: Transform to the Frequency Domain

The noisy signal is first transformed using the implemented DFT or FFT algorithm. The resulting spectrum reveals the magnitude distribution over discrete frequency bins. This stage should be accompanied by:

- waveform plot of the input signal in the time domain;
- magnitude spectrum of the noisy signal;
- brief interpretation of dominant peaks or suspicious noise regions.

5.3. Step 2: Identify and Attenuate Noise Components

Depending on the nature of the recorded audio, the filtering rule can be described as one of the following:

- threshold-based suppression of small spectral components;
- notch removal of narrow-band hum;
- band attenuation of clearly noisy frequency ranges;
- frame-wise masking if short-time processing is used.

For this experiment, magnitude-threshold filtering is selected because the meaningful tonal components of the test signal create large spectral magnitudes, while many noise-dominated bins have relatively small magnitudes spread across the spectrum. After computing $|X_k|$, the algorithm keeps only frequency components whose magnitudes are at least 15% of the maximum magnitude in the current frame and sets the remaining coefficients to zero. This rule follows directly from the interpretation of $|X_k|$ as component strength and is suitable for demonstrating the main idea of frequency-domain denoising. Although it is not as adaptive as advanced speech enhancement methods, it clearly shows how spectral information can be used to suppress noise before signal reconstruction.

5.4. Step 3: Signal Reconstruction

After spectral modification, the inverse DFT is applied to reconstruct the denoised audio signal. The phase of the retained coefficients is preserved because phase information is necessary for reconstructing the waveform shape correctly. Only the selected magnitudes are suppressed through thresholding. This is important: removing magnitude information changes the strength of frequency components, while preserving phase helps maintain temporal alignment.

The reconstructed chunks are multiplied by the same Hanning window and added back into the output buffer. Since overlapping windows contribute to the same sample positions, the program also accumulates the squared window weights in `window_sum`. Dividing by `window_sum` normalizes the final waveform and prevents overlapped regions from becoming artificially louder or quieter. The final output is then converted back to integer audio samples and saved as `sample_cleaned_smooth.wav`.

5.5. Expected Figures and Tables

To satisfy the assignment clearly, the final report should include at least the following items:

1. waveform before denoising;
2. waveform after denoising;
3. frequency spectrum before filtering;
4. frequency spectrum after filtering;
5. short explanation of the denoising effect and any remaining artifacts.

6. Experimental Results Template

6.1. Experiment Setup

6.2. Program Output and Visualization

Figure 1 shows the program execution after running the denoising script. The script reads the noisy audio input, processes the signal using the DFT-based denoising pipeline, and saves the reconstructed audio as `sample_cleaned_smooth.wav`.

Table 3: Suggested experiment summary table

Item	Description
Programming language	Python
Libraries used	`numpy` for numerical arrays, `scipy.io.wavfile` for audio I/O, and `matplotlib` for plotting
Core transform method	Direct DFT and inverse DFT implemented from formulas
Audio duration	Approximately 1.20 seconds
Windowing strategy	Block-wise processing with 1024-sample Hanning windows and 50% overlap-add
Filtering strategy	Magnitude thresholding with threshold ratio 0.15

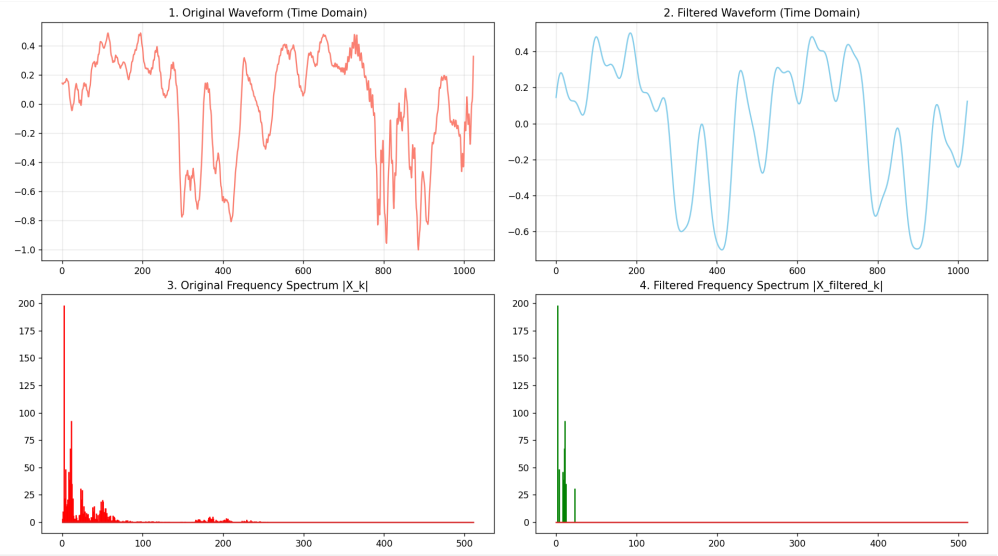


Figure 1: Program execution screenshot for the DFT-based audio denoising script.

Figure 2 presents the required plots after running the code. The upper row compares the original and filtered waveforms in the time domain, while the lower row compares the original and filtered magnitude spectra in the frequency domain.

Before running the experiment, the expected result was not a perfectly clean audio signal. Since the method uses a fixed magnitude threshold, it is designed mainly to reduce weaker spectral components rather than to perform intelligent source separation. Therefore, the expected output was a smoother waveform, a sparser spectrum, and preservation of the most dominant tonal components. The result in Figure 2 is consistent with this expectation: the filtered spectrum keeps the strongest peaks while removing many small components.

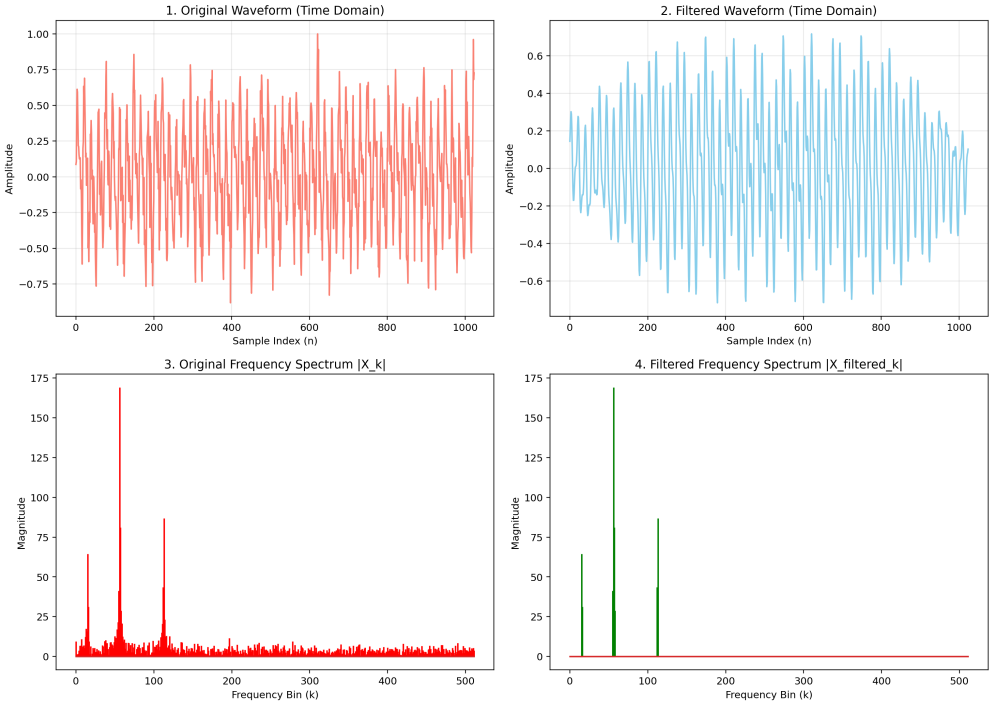


Figure 2: Waveform and frequency-spectrum comparison before and after denoising.

Filtered audio result: The final denoised audio file can be accessed through the following Google Drive link: Filtered Audio Output. This link is provided as listening evidence to complement the waveform and spectrum plots above.

6.3. Result Discussion

The plotted spectra show that the original signal contains several dominant peaks together with many small-amplitude components distributed across the frequency bins. The dominant peaks correspond to the main tonal structure of the signal, while the weaker distributed components represent noise-like energy. After thresholding, the filtered spectrum becomes much sparser: the principal peaks are preserved, whereas many low-magnitude bins are removed.

In the time domain, the filtered waveform appears smoother and less irregular than the noisy waveform. This indicates that a significant portion of high-frequency and low-energy fluctuations has been suppressed. The result confirms the central idea of the DFT-based method: once the signal is represented in the frequency domain, noise components can be attenuated by modifying the spectral coefficients before applying the inverse transform.

The method also has limitations. A fixed threshold may remove weak but meaningful signal components if their magnitudes are close to the noise floor. Conversely, strong noise components may remain if their magnitudes exceed the threshold. Therefore, this implementation is appropriate for illustrating the mathematical mechanism of DFT-based denoising, while more advanced practical systems would require adaptive thresholds, frame-wise noise estimation, or perceptual filtering rules.

Compared with the expected behavior, the observed result is reasonable. The frequency-domain plot after filtering is not empty, meaning that the algorithm did not destroy the whole signal. It retains a small number of high-energy bins, which correspond to the main periodic structure. At the same time, the reduction of many small bins indicates that the thresholding rule is doing its intended job. In the time-domain plot, the filtered waveform is still recognizable as an oscillatory signal but has fewer irregular fluctuations. This supports the conclusion that the algorithm reduces part of the noise while preserving the main content.

One noteworthy observation is that visual smoothness does not automatically mean perceptual perfection. A waveform may look cleaner while still containing audible artifacts, especially if spectral components are removed too abruptly. This is why the overlap-add method is important: it reduces artificial clicks caused by chunk boundaries. The denoising rule affects the spectrum inside each chunk, while overlap-add improves the continuity between chunks. The final quality is therefore the combined result of both mathematical filtering and careful reconstruction.

6.4. Comparison Between DFT-Based and FFT-Based Results

The two implementations are expected to produce similar denoising behavior because the FFT is only a faster algorithm for computing the DFT. In both versions, the noisy signal is transformed into spectral coefficients, low-magnitude components are attenuated, and the inverse transform reconstructs the audio. Thus, the meaning of X_k, $|X_k|$, and the thresholding rule remains unchanged.

The practical difference is efficiency. The DFT version is useful for learning because it follows the formula directly, making the summation structure visible. The FFT version is more suitable for larger signals because it avoids redundant computation through recursive even–odd splitting. Any small difference in the plotted or audible result is mainly caused by implementation parameters such as the threshold ratio, not by a

different mathematical transform.

6.5. Evaluation Note

The experiment is evaluated through both visual and algorithmic observations:

- In the frequency spectrum, low-magnitude noise bins are visibly reduced after filtering.
- In the waveform, the reconstructed signal is smoother and less dominated by random oscillations.
- The main spectral peaks are preserved, which means the principal signal components remain after denoising.
- The output audio file `sample_cleaned_smooth.wav` provides an auditory check of the reconstruction quality.
- Remaining artifacts are expected because the threshold is fixed and does not adapt to local noise conditions.

6.6. Project Deliverables

The final deliverables of this project include the written report, the implementation source code, the input and output audio files, and the visual evidence generated from the experiment. They are listed as follows:

- **Report file:** The final PDF report generated from the LaTeX source code.
- **Source code:** A Python script implementing manual DFT, manual inverse DFT, magnitude-based denoising, Hanning windowing, and overlap-add reconstruction.
- **Input audio file:** The noisy audio file `sample.wav`, used as the input signal for the denoising experiment.
- **Output audio file:** The reconstructed denoised audio file `sample_cleaned_smooth.wav`.
- **Visualization images:** Waveform and frequency-spectrum comparison before and after denoising, including `figures/denoising_results.png`.

- **Program execution evidence:** A screenshot showing successful program execution, saved as `figures/code_run_screenshot.png`.
- **Executable file:** No standalone executable file is produced. The program is executed directly through the Python script.

These deliverables show that the project is not limited to theoretical discussion. The group also implemented the algorithm, generated experimental outputs, and prepared supporting evidence for evaluation and presentation.

7. Conclusion and Future Work

This project shows that the DFT provides a mathematically rigorous bridge between the time-domain representation of an audio signal and its spectral structure in the frequency domain. The FFT then makes this transformation computationally practical for realistic signals. Even though the implementation in this report intentionally computes the DFT directly from the formula, the study of FFT complexity explains why optimized algorithms are essential in real applications.

From the perspective of linear algebra, the DFT is a linear transformation acting on a finite-dimensional complex vector space. From the perspective of signal processing, it enables frequency-domain filtering, which is especially useful for audio denoising because noise components can often be isolated more effectively in the spectrum than in the raw waveform. The project therefore connects abstract mathematical concepts with a concrete engineering task: improving the quality of an audio signal.

The practical part of the project also revealed several difficulties. The first difficulty was computational cost. A direct DFT is clear and faithful to the formula, but its $O(N^2)$ complexity makes it slow for long recordings. The group addressed this issue by dividing the audio into chunks, so each DFT computation remained small enough to run. The second difficulty was reconstruction quality. Processing independent chunks can introduce clicking artifacts at the boundaries. To improve this, the implementation uses Hanning windows and overlap-add reconstruction, which makes the transition between chunks smoother.

Another difficulty was choosing a filtering threshold. If the threshold is too low, much noise remains. If it is too high, useful signal components may disappear. The selected

threshold ratio of 0.15 is a simple compromise that demonstrates the idea clearly, but it is not universally optimal. This limitation helped the group understand that audio denoising is not only a formulaic task; it also requires experimental judgment, parameter tuning, and interpretation of both visual and auditory results.

Overall, the project achieved its main goal: it built the theoretical path from Fourier analysis to DFT/FFT, implemented the transform directly from the mathematical definition, and applied it to an audio denoising pipeline. More importantly, it showed how linear algebra can become a practical computational tool. A noisy waveform is transformed into a vector of spectral coefficients, selected components are modified, and the final signal is reconstructed from the inverse transformation.

Future directions may include short-time Fourier analysis with more systematic frame processing, adaptive threshold selection based on estimated noise floors, spectral subtraction using a noise-only segment, comparison with wavelet-based denoising, or combining Fourier-domain filtering with matrix methods such as SVD. These extensions would make the system more robust while preserving the same fundamental idea: representing a signal in a domain where useful structure and noise are easier to distinguish.

Lessons Learned from the Project

A major lesson from the project is that a mathematically correct formula is not automatically a complete engineering solution. The DFT formula is exact, but using it directly on a long signal creates a heavy computational burden. The team therefore had to balance mathematical transparency with practical execution. Chunking the signal made the computation possible, while overlap-add reconstruction solved the discontinuity problem introduced by chunking.

Another lesson is that audio denoising requires interpretation. The spectrum does not label components as "signal" or "noise" by itself. The group had to decide a threshold, observe the plotted result, listen to the output, and reason about what the algorithm preserved or removed. This process made the project more than a coding exercise: it became an experiment in connecting theory, implementation, visualization, and evaluation.

Finally, the project showed that linear algebra is not isolated from technology. Matrix transformations, complex vector spaces, and basis changes can directly influence how

a real audio file is processed. This connection is the main value of the report: it demonstrates how a core mathematical subject can become a practical tool for signal analysis and digital audio improvement.

Appendix: Requirement Checklist

The report satisfies the core requirements of the assignment: it develops the theoretical background of Fourier analysis, derives and interprets the DFT formula, explains the FFT idea and computational motivation, implements the direct DFT/IDFT formulas, and demonstrates waveform and spectrum comparisons before and after denoising.

References

[1] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*. Pearson, 3rd edition, 2009.

[2] G. Strang, *Introduction to Linear Algebra*. Wellesley-Cambridge Press, 5th edition, 2016.

[3] Đặng Thị Kim Nhung, Ph.D. *Linear Algebra (MT1007) Course Materials*, Ho Chi Minh City University of Technology, 2025–2026.

[4] *Discrete Fourier Transform (DFT–FFT) and Audio Denoising*, major project assignment handout, 2026.